

# Universidade de Vigo

Memoria Proyecto (Primera Parte) - Sistemas Inteligentes

## Warehouse

Grupo SI 8-2

Luis Garbayo Fernández  
Yerai Fernández Rodríguez  
Alberto Andrés Faublak Álvarez  
Vincenzo Gagliano  
Eva Barreiro Ferreira

2025–2026

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA



# Índice general

|                                                           |          |
|-----------------------------------------------------------|----------|
| <b>1. Introducción</b>                                    | <b>1</b> |
| <b>2. Antecedentes y contexto</b>                         | <b>2</b> |
| 2.1. Herramientas y tecnologías existentes . . . . .      | 2        |
| 2.1.1. El modelo BDI y AgentSpeak . . . . .               | 2        |
| 2.1.2. La plataforma Jason . . . . .                      | 3        |
| 2.1.3. La metodología Prometheus . . . . .                | 3        |
| 2.1.4. El patrón Modelo-Vista-Controlador . . . . .       | 3        |
| 2.1.5. Algoritmo de búsqueda de rutas BFS . . . . .       | 4        |
| 2.2. Conclusión . . . . .                                 | 4        |
| <b>3. Objetivos</b>                                       | <b>5</b> |
| 3.1. Objetivo General . . . . .                           | 5        |
| 3.2. Objetivos específicos . . . . .                      | 5        |
| 3.2.1. Primera semana (objetivos iniciales) . . . . .     | 5        |
| 3.2.2. Segunda semana (objetivos incrementales) . . . . . | 6        |
| <b>4. Diseño del SMA</b>                                  | <b>7</b> |
| 4.1. Diagrama de entorno . . . . .                        | 7        |
| 4.2. Diagrama de organización . . . . .                   | 8        |
| 4.3. Diagrama de objetivos ideal . . . . .                | 9        |
| 4.4. Diagrama de objetivos . . . . .                      | 10       |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 4.5. Diagrama de interacción . . . . .                        | 11        |
| 4.6. Diagrama de agente y tareas . . . . .                    | 12        |
| 4.6.1. Agentes robot (light, medium, heavy) . . . . .         | 12        |
| 4.6.2. Agente scheduler . . . . .                             | 17        |
| 4.6.3. Agente supervisor . . . . .                            | 19        |
| <b>5. Arquitectura: Patrón MVC (Modelo-Vista-Controlador)</b> | <b>21</b> |
| 5.1. El Modelo: datos y estado del almacén . . . . .          | 21        |
| 5.2. La Vista: interfaz gráfica . . . . .                     | 21        |
| 5.3. El Controlador: artefacto y agentes . . . . .            | 22        |
| <b>6. Implementación de la solución</b>                       | <b>24</b> |
| 6.1. El Entorno Java . . . . .                                | 24        |
| 6.2. El Agente Scheduler . . . . .                            | 25        |
| 6.3. Los Agentes Robot . . . . .                              | 26        |
| 6.4. El Agente Supervisor . . . . .                           | 28        |
| 6.5. Robustez y Gestión de Fallos . . . . .                   | 29        |
| <b>7. Pruebas y Resultados</b>                                | <b>30</b> |
| 7.1. Escenario base: operación nominal . . . . .              | 30        |
| 7.2. Escenario de aplastamiento de contenedor . . . . .       | 30        |
| 7.3. Escenario de saturación del almacén . . . . .            | 31        |
| 7.4. Validación del agente supervisor . . . . .               | 31        |
| <b>8. Conclusiones y trabajo futuro</b>                       | <b>32</b> |

# Índice de figuras

|                                                       |    |
|-------------------------------------------------------|----|
| 4.1. Diagrama de entorno del sistema. . . . .         | 8  |
| 4.2. Diagrama de organización del sistema. . . . .    | 9  |
| 4.3. Diagrama de objetivos ideal del sistema. . . . . | 10 |
| 4.4. Diagrama de objetivos del sistema. . . . .       | 11 |
| 4.5. Diagrama de interacción del sistema. . . . .     | 12 |
| 4.6. Diagrama de agente: robot light. . . . .         | 13 |
| 4.7. Diagrama de tareas: robot light. . . . .         | 14 |
| 4.8. Diagrama de agente: robot medium. . . . .        | 15 |
| 4.9. Diagrama de tareas: robot medium. . . . .        | 15 |
| 4.10. Diagrama de agente: robot heavy. . . . .        | 16 |
| 4.11. Diagrama de tareas: robot heavy. . . . .        | 17 |
| 4.12. Diagrama de agente: scheduler. . . . .          | 18 |
| 4.13. Diagrama de tareas: scheduler. . . . .          | 18 |
| 4.14. Diagrama de agente: supervisor. . . . .         | 19 |
| 4.15. Diagrama de tareas: supervisor. . . . .         | 20 |

# 1. Introducción

La gestión autónoma de almacenes logísticos plantea un problema de coordinación inherentemente complejo: múltiples agentes físicos con capacidades heterogéneas deben operar sobre un espacio compartido, tomando decisiones en tiempo real sin intervención humana directa. La dificultad no reside únicamente en el movimiento individual de cada robot, sino en la coordinación emergente del conjunto: evitar conflictos de acceso, distribuir la carga de trabajo de forma eficiente, reaccionar ante fallos imprevistos y garantizar la correcta gestión de mercancía.

Los Sistemas Multiagente (SMA) constituyen un paradigma especialmente adecuado para abordar este tipo de problemas. Su capacidad para modelar entidades autónomas que perciben el entorno, razonan sobre sus objetivos y coordinan su comportamiento mediante paso de mensajes permite descomponer la complejidad global del sistema en agentes individuales con responsabilidades bien definidas. En particular, el modelo de agente BDI (*Beliefs, Desires, Intentions*) proporciona una base formal sólida para especificar el comportamiento reactivo y deliberativo de cada entidad, facilitando tanto el diseño como el análisis del sistema resultante.

Este proyecto parte de una base de código proporcionada por la asignatura y la extiende sustancialmente: realizando modificaciones sobre los agentes, el entorno Java y la interfaz gráfica hasta obtener un sistema funcional y capaz de gestionar la logística completa del almacén. El diseño de los agentes y sus interacciones se documenta mediante los artefactos de la metodología Prometheus (diagramas de entorno, organización, objetivos, interacción y tareas), que permiten una especificación formal y sistemática del SMA antes de su implementación en Jason. Asimismo, la infraestructura Java adopta el patrón Modelo-Vista-Controlador (MVC) para separar de forma clara la lógica de negocio, la representación visual y el control físico del entorno.

El sistema resultante coordina de forma autónoma el transporte de contenedores desde la zona de entrada del almacén hasta las estanterías de almacenamiento, haciendo uso de tres tipologías de robots con capacidades diferenciadas (*light, medium* y *heavy*), un agente planificador central que actúa como cerebro decisional del sistema (*scheduler*), y un agente supervisor (*supervisor*) que audita el estado global del almacén en tiempo real.

## 2. Antecedentes y contexto

Los sistemas multiagente llevan décadas evolucionando desde sus bases teóricas hasta plataformas concretas usadas tanto en investigación como en industria. Lo que empezó como un formalismo para describir el comportamiento de agentes computacionales ha derivado en plataformas maduras capaces de coordinar múltiples agentes heterogéneos sobre entornos físicos reales. En el centro de esta evolución se sitúa el paradigma BDI (*Beliefs, Desires, Intentions*), que proporciona un marco cognitivo para que los agentes razonen sobre su conocimiento del mundo, sus objetivos y los compromisos de acción adquiridos para alcanzarlos. El presente capítulo sitúa el proyecto en ese contexto: describe las herramientas y paradigmas sobre los que se construye, y revisa los sistemas y estudios previos que justifican las decisiones de diseño adoptadas.

### 2.1. Herramientas y tecnologías existentes

#### 2.1.1. El modelo BDI y AgentSpeak

El paradigma de agente predominante en el ámbito de los sistemas multiagente deliberativos es el modelo BDI (*Beliefs, Desires, Intentions*), formalizado por Rao y Georgeff en 1995 [1]. En este modelo, cada agente mantiene tres estructuras mentales diferenciadas: sus creencias (*beliefs*), que representan el conocimiento que el agente tiene sobre el mundo; sus deseos (*desires*), que modelan los estados del mundo que el agente querría alcanzar; y sus intenciones (*intentions*), que son los compromisos de acción que el agente ha adoptado para satisfacer sus deseos. La ventaja del modelo BDI sobre enfoques puramente reactivos o estrictamente deliberativos radica en su capacidad para combinar reactividad ante eventos del entorno con planificación orientada a objetivos a largo plazo.

AgentSpeak, introducido por Rao en 1996 [2], es el lenguaje de programación de referencia para agentes BDI. Su sintaxis se basa en planes compuestos por un evento desencadenante, un contexto de aplicabilidad y un cuerpo de acciones. La semántica operacional de AgentSpeak establece de forma precisa cómo se seleccionan y ejecutan los planes ante la llegada de eventos, y cómo se gestionan los fallos mediante planes de contingencia negativos.

### 2.1.2. La plataforma Jason

Jason [3] [4] es la implementación de referencia de AgentSpeak. Proporciona un intérprete del ciclo de razonamiento BDI, soporte para comunicación entre agentes mediante paso de mensajes asíncronos, y una API Java que permite conectar los agentes con entornos externos implementados en dicho lenguaje. Esta última característica es especialmente relevante para el presente proyecto: la clase `jason.environment.Environment` actúa como puente entre la lógica de los agentes `.asl` y el artefacto Java `WarehouseArtifact`, que implementa la física del almacén.

Jason se ejecuta sobre la máquina virtual de Java, lo que garantiza portabilidad, y dispone de herramientas de depuración integradas como el *Agent Mind Inspector*, que permite inspeccionar en tiempo real la base de creencias, las intenciones activas y el ciclo de razonamiento de cada agente durante la ejecución, y ha sido de gran ayuda a la hora de la implementación del proyecto y de posibles errores detectados en ella.

### 2.1.3. La metodología Prometheus

Prometheus [6] es una metodología de ingeniería de software específicamente diseñada para el desarrollo de SMA. Proporciona un proceso estructurado que guía al desarrollador desde la especificación del sistema hasta su implementación, pasando por el diseño de agentes e interacciones.

En el presente proyecto, Prometheus se ha empleado para especificar formalmente los cinco agentes del sistema, sus percepciones, acciones y protocolos de comunicación, tal como se especificó en clase y como se documenta en capítulos posteriores.

### 2.1.4. El patrón Modelo-Vista-Controlador

El patrón arquitectónico MVC (*Model-View-Controller*) se ha adoptado en la capa Java del sistema para separar claramente tres responsabilidades: el modelo (clases `Container`, `Robot`, `Shelf`, `CellType`) almacena el estado del almacén; la vista (`WarehouseView`) renderiza dicho estado gráficamente mediante Java Swing; y el controlador (`WarehouseArtifact`) recibe las acciones de los agentes, modifica el modelo y notifica a la vista. Esta separación facilita el mantenimiento del código y permite modificar cualquiera de las tres capas sin afectar a las otras dos.



### 2.1.5. Algoritmo de búsqueda de rutas BFS

La navegación de los robots en el almacén se resuelve mediante búsqueda en anchura (*Breadth-First Search*, BFS) sobre la rejilla de celdas. BFS garantiza encontrar el camino más corto en grafos no ponderados, lo que es adecuado para una rejilla donde todos los movimientos tienen el mismo coste. El algoritmo considera como obstáculos las celdas de tipo SHELF y BLOCKED, así como las posiciones ocupadas por otros robots en ese instante, implementando una forma básica de evasión dinámica de colisiones.

## 2.2. Conclusión

BDI, Jason y Prometheus son opciones bien establecidas y documentadas para este tipo de sistemas; su uso en el proyecto no es arbitrario sino que responde directamente a los requisitos de la asignatura y a la naturaleza del problema que se quiere resolver.

## 3. Objetivos

### 3.1. Objetivo General

El objetivo general de este proyecto es diseñar e implementar un sistema multiagente (SMA) capaz de gestionar de forma autónoma las operaciones logísticas de un almacén simulado. El sistema debe coordinar varios agentes BDI, con roles diferenciados de planificación, ejecución y supervisión, para clasificar contenedores, asignar tareas a robots según sus capacidades y almacenar los contenedores en las estanterías correctas, gestionando además las situaciones de error que puedan surgir durante la operación.

### 3.2. Objetivos específicos

#### 3.2.1. Primera semana (objetivos iniciales)

- **El *scheduler* detecta nuevos contenedores.** El agente planificador debe ser capaz de percibir en tiempo real la llegada de nuevos contenedores al almacén e iniciar su procesamiento de forma autónoma.
- **El *scheduler* asigna el contenedor a un robot.** El planificador debe seleccionar, para cada contenedor, el robot más adecuado en función de sus características, garantizando una distribución de tareas coherente con las capacidades de cada robot.
- **El *scheduler* asigna estantería al contenedor.** Cada contenedor debe tener una estantería destino reservada antes de ser transportado, evitando conflictos de almacenamiento.
- **El robot realiza la tarea asignada.** Los robots deben ejecutar de forma autónoma las tareas que les asigne el *scheduler*, sin intervención externa durante el proceso.
- **El robot recoge el contenedor y lo lleva a la estantería.** El ciclo completo, desde la recogida del contenedor hasta su depósito en la estantería, debe ser ejecutado íntegramente por el robot.
- **El robot encuentra el camino a la estantería.** Los robots deben ser capaces de calcular rutas

válidas por el almacén para alcanzar tanto los contenedores como las estanterías destino.

- **Todos los robots llevan contenedores a estanterías.** Los tres tipos de robot (ligero, mediano y pesado) deben ser operativos y capaces de completar transportes de forma simultánea e independiente.
- **Los robots manejan errores de asignación.** Ante situaciones de error durante la ejecución de una tarea, los robots deben gestionarlas adecuadamente y notificar al resto del sistema para permitir la recuperación.
- **El supervisor monitoriza contenedores recibidos, almacenados y errores.** El agente supervisor debe mantener un registro actualizado del estado de todos los contenedores que circulan por el almacén.
- **El supervisor calcula estadísticas.** El supervisor debe producir periódicamente informes cuantitativos sobre el rendimiento global del sistema.

### 3.2.2. Segunda semana (objetivos incrementales)

- **El scheduler recibe contenedores.** El planificador debe suscribirse a las notificaciones del entorno para detectar cada nuevo contenedor en el momento de su llegada.
- **El scheduler clasifica el contenedor por peso, tamaño y tipo.** Cada contenedor debe ser categorizado según tres dimensiones independientes: peso, dimensiones y tipo de carga, para guiar las decisiones de asignación posteriores.
- **El scheduler sitúa los contenedores en puntos distintos.** Los contenedores deben distribuirse entre varias posiciones de entrada del almacén, evitando que todos se acumulen en el mismo punto.
- **El scheduler asigna la estantería en la que almacenar el contenedor clasificado.** La elección de estantería debe tener en cuenta la clasificación previa del contenedor, agrupando cada tipo en el área de almacenamiento correspondiente.
- **El scheduler mantiene la trazabilidad robot–contenedor–estantería.** El planificador debe registrar en todo momento qué robot es responsable de cada contenedor y a qué estantería está destinado.
- **El supervisor monitoriza el estado de los robots.** El supervisor debe conocer en todo momento si cada robot está activo o en espera, reflejando los cambios de estado a medida que se producen.

## 4. Diseño del SMA

A continuación se presentan los diagramas resultantes de aplicar esta metodología al proyecto *Warehouse* [5]. Cada diagrama captura una dimensión distinta del sistema y sirve como especificación formal previa a la implementación. Los diagramas de agente y tareas se incluyen individualmente para cada uno de los cinco agentes del sistema, detallando sus creencias iniciales, las percepciones que reciben del entorno y los planes que ejecutan.

### 4.1. Diagrama de entorno

El entorno del sistema, como se ilustra en la Figura 4.1, está implementado en la clase Java *WarehouseArtifact*, que actúa como interfaz entre los agentes *AgentSpeak* y la lógica física del almacén. Los agentes no acceden directamente al estado interno del almacén: todo cambio en el mundo físico se comunica a través de percepciones que el entorno emite, y toda acción física se ejecuta mediante llamadas al entorno.

Las percepciones que el entorno puede emitir a los agentes son las siguientes:

- `new_container(CId)`: emitida globalmente al aparecer un nuevo contenedor en la zona de entrada del almacén.
- `container_info(CId, W, H, Weight, Type, X, Y)`: describe las propiedades físicas del contenedor: anchura, altura, peso, tipo y posición actual en el mapa.
- `free_shelf(CId, ShelfId)`: confirma la estantería reservada para el contenedor indicado.
- `robot_at(X, Y)`: refleja la posición actualizada del robot tras cada movimiento.
- `picked`: confirma que el robot ha recogido físicamente el contenedor asignado.
- `stored(CId, ShelfId)`: confirma que el contenedor ha sido depositado correctamente en la estantería destino.
- `state(Status)`: indica el estado operacional del robot en cada momento.
- `error(ErrorType, Data)`: emitida cuando ocurre una anomalía física. Los tipos posibles in-

cluyen `destination_conflict`, `route_blocked`, `path_blocked`, `too_far`, `illegal_move` y `robot_not_found`.

Las acciones que los agentes pueden ejecutar sobre el entorno se agrupan en tres categorías: navegación (`move_to_container(CId)`, `move_to_shelf(ShelfId)`), manipulación de carga (`pickup(CId)`, `drop_at(ShelfId)`) y consultas al controlador (`get_container_info(CId)`, `get_free_shelf(CId)`, `accept_task(CId)`, `release_task`).

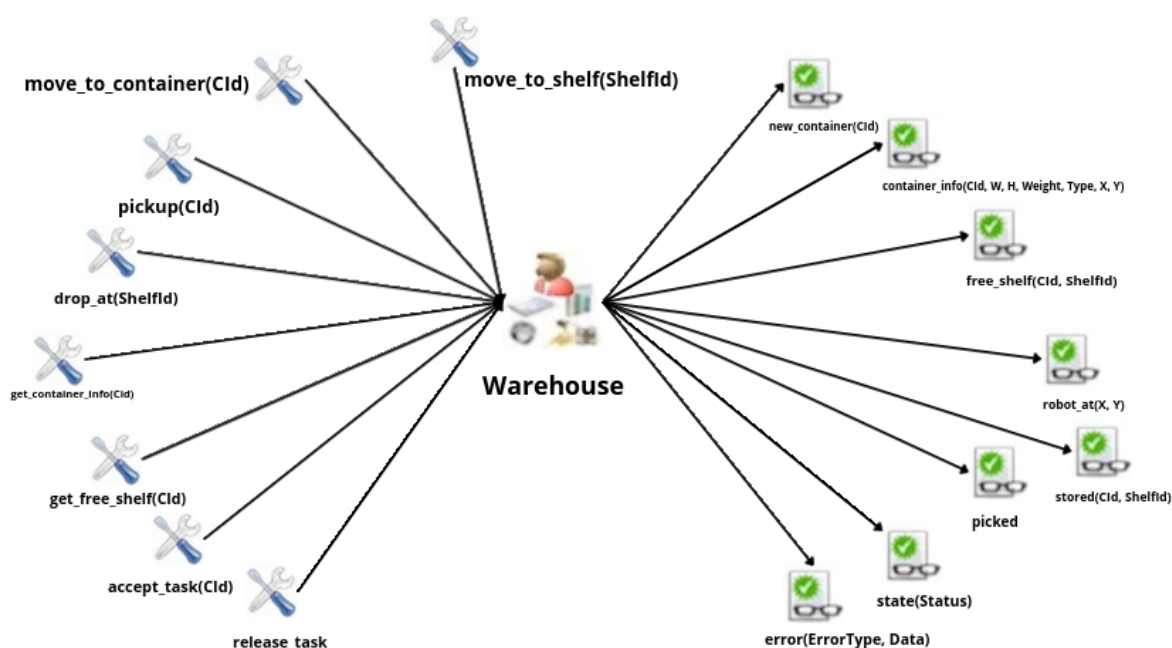


Fig. 4.1. Diagrama de entorno del sistema.

## 4.2. Diagrama de organización

El diagrama de organización (ver Figura 4.2) muestra la topología centralizada del SMA, definida en el fichero de configuración `warehouse.mas2j`. La estructura se organiza en dos capas de responsabilidad diferenciadas:

- **Capa de control lógico:** formada por el agente planificador (*scheduler*), que actúa como cerebro decisional del almacén gestionando la clasificación de contenedores y la asignación de tareas, y el agente auditor (*supervisor*), que monitoriza el sistema de forma pasiva sin intervenir en la planificación.
- **Capa de ejecución física:** formada por tres agentes heterogéneos (*robot\_light*, *robot\_medium*,

robot\_heavy) que ejecutan materialmente el transporte de contenedores, cada uno con capacidades operativas distintas en cuanto a peso y dimensiones máximas admisibles.

Los robots nunca se comunican directamente entre sí. Mantienen comunicación bidireccional con el *scheduler* —reciben asignaciones de tareas y le reportan éxitos (*container\_stored*) o fallos (*task\_failed*)— y comunicación unidireccional con el *supervisor*, al que notifican cambios de estado y errores operativos. El *supervisor* consulta el estado de los robots en tiempo real durante cada ciclo de reporte, sin emitir órdenes.

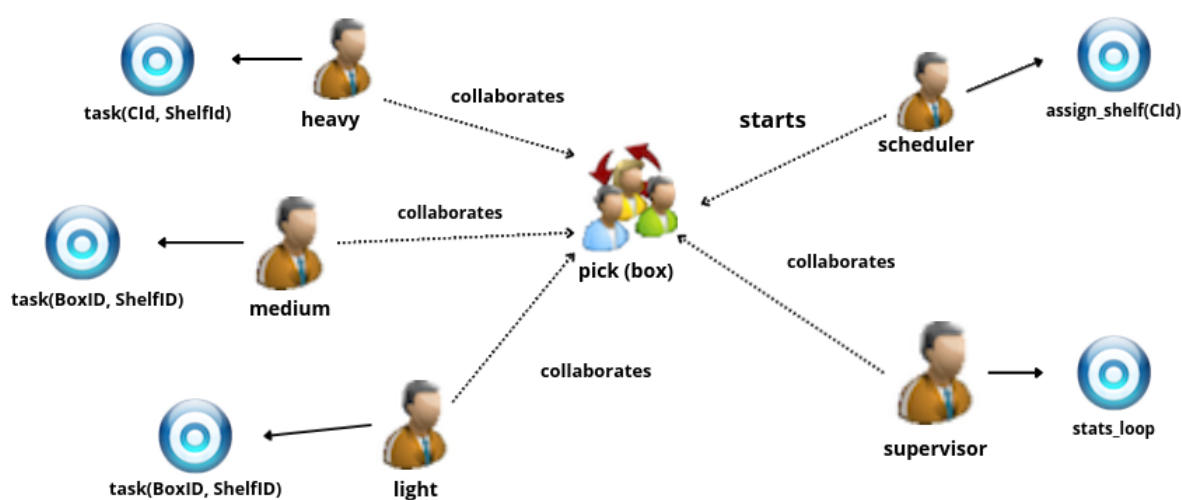


Fig. 4.2. Diagrama de organización del sistema.

### 4.3. Diagrama de objetivos ideal

El diagrama de objetivos ideal (ver Figura 4.3) ilustra el flujo completo del sistema en condiciones normales, sin errores ni interrupciones, desde la aparición de un nuevo contenedor hasta su almacenamiento definitivo. El ciclo comienza cuando el entorno emite la percepción *new\_container(CId)*, lo que activa en el *scheduler* el objetivo *get\_container\_info*, mediante el cual el planificador consulta al entorno las propiedades físicas del contenedor recién llegado. Con esa información, el *scheduler* resuelve el objetivo *assign\_shelf*, que reserva la estantería adecuada según la categoría del contenedor. Una vez completada la asignación, el robot seleccionado recibe la tarea y lanza el objetivo *execute\_task*, que se descompone bajo nodo AND en cuatro fases que deben completarse en su totalidad: *move\_to\_container*, que desplaza al robot hasta el contenedor; *pickup*, que lo recoge físicamente; *move\_to\_shelf*, que lo transporta hasta la estantería reservada; y *drop\_at*, que completa el depósito. En paralelo, el *supervisor* ejecuta de forma iterativa *print\_stats*, regis-

trando y publicando por consola el estado global del sistema tras cada ciclo completado.

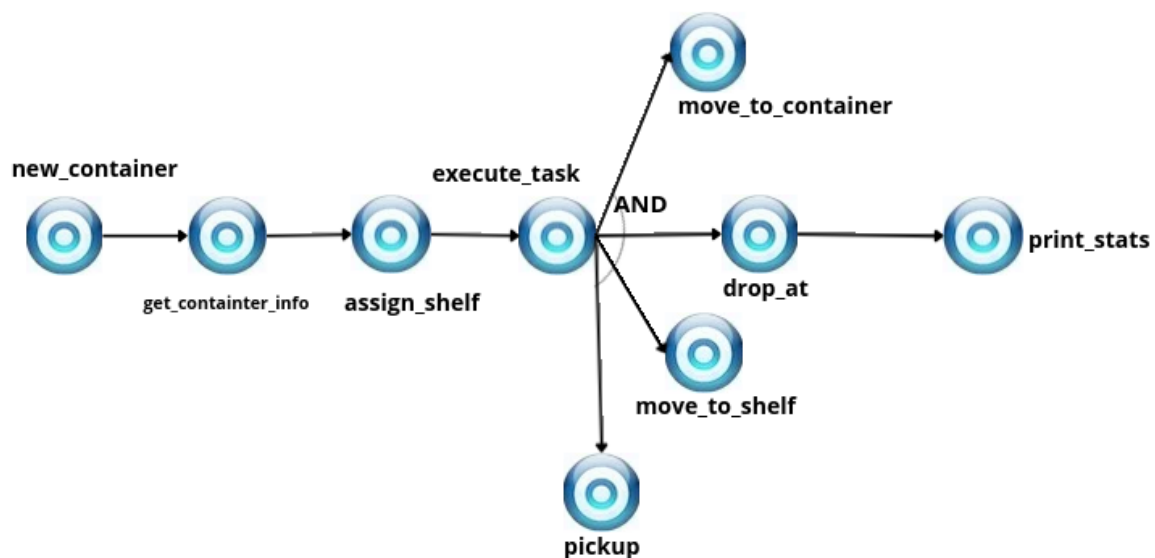


Fig. 4.3. Diagrama de objetivos ideal del sistema.

#### 4.4. Diagrama de objetivos

El diagrama de objetivos (ver Figura 4.4) descompone en mayor detalle los tres objetivos principales del sistema. El objetivo `assign_shelf` del `scheduler` tiene como único sub-objetivo `get_free_shelf`, encargado de consultar y reservar la estantería correspondiente según la categoría del contenedor. El objetivo `execute_task` de los robots se descompone en cuatro sub-objetivos bajo nodo AND: `move_to_container`, `pickup`, `move_to_shelf` y `drop_at`, indicando que todos deben completarse para considerar la tarea satisfecha. Finalmente, el objetivo `stats_loop` del supervisor desencadena `print_stats`, que a su vez se apoya en tres sub-objetivos auxiliares: `print_errors_by_type`, `print_error_list` y `print_robot_status`.

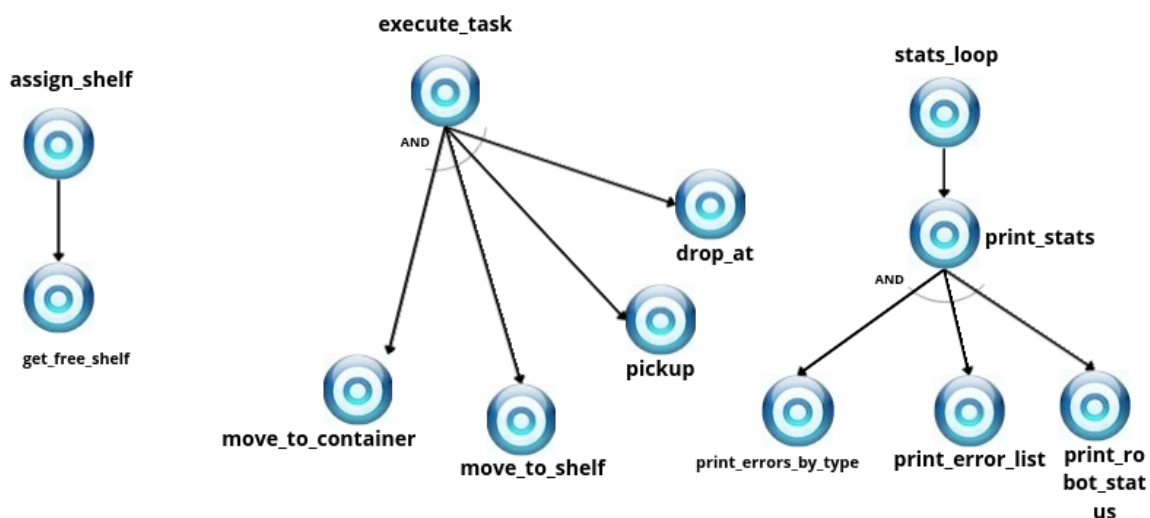


Fig. 4.4. Diagrama de objetivos del sistema.

## 4.5. Diagrama de interacción

El diagrama de interacción (ver Figura 4.5) define los protocolos de paso de mensajes entre los agentes del SMA. La interacción central se inicia cuando el entorno emite la percepción `new_container(CId)`, que es recibida directamente por el *scheduler*.

A partir de ese momento, el *scheduler* selecciona el robot adecuado y le envía el mensaje `tell: task(CId, ShelfId)`, añadiendo la tarea como creencia en la base de conocimiento del robot receptor. El uso del performativo `tell` refleja que el *scheduler* no impone un objetivo al robot, sino que actualiza su estado de creencias, siendo el propio robot quien reacciona de forma autónoma al detectar la nueva creencia. Una vez recibida la tarea, el robot ejecuta sobre el entorno la secuencia de acciones físicas `move_to_container(CId) → pickup(CId) → move_to_shelf(ShelfId) → drop_at(ShelfId)`, que constituyen el ciclo completo de transporte.

Al concluir la tarea, el robot notifica el resultado simultáneamente al *scheduler* y al *supervisor*. Si la tarea tuvo éxito, envía `tell: container_stored(CId, ShelfId)` a ambos. Si se produjo un fallo durante la ejecución, envía `tell: task_failed(CId)` al *scheduler* para que reasigne el contenedor, y `tell: container_error(CId, ErrorType)` o `tell: robot_error(Robot, ErrorType, Data)` al *supervisor* para su registro.

El *supervisor* es el único agente del sistema que no emite mensajes hacia otros agentes: su rol es exclusivamente de auditor, recibiendo notificaciones de los robots y percepciones del entorno para



calcular y publicar estadísticas periódicas.

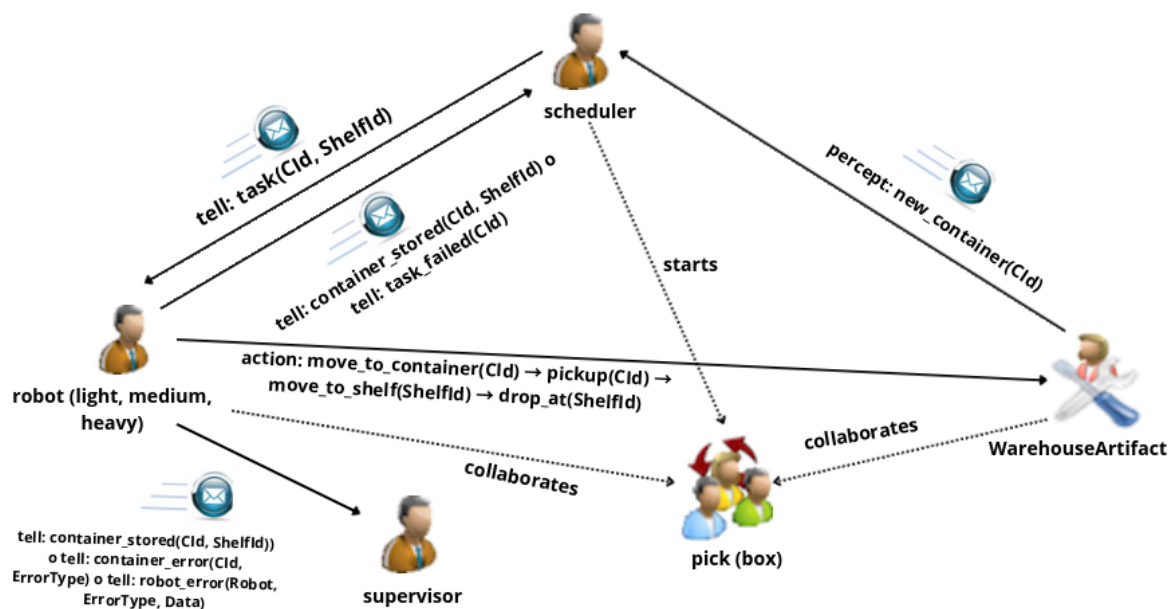


Fig. 4.5. Diagrama de interacción del sistema.

## 4.6. Diagrama de agente y tareas

A continuación se describe la arquitectura cognitiva de cada agente del sistema, incluyendo sus creencias iniciales, las percepciones que reciben del entorno y los planes que ejecutan. Los diagramas de agente y de tareas correspondientes se adjuntan en cada subsección.

### 4.6.1. Agentes robot (light, medium, heavy)

Los tres agentes robot comparten la misma arquitectura base de comportamiento, diferenciándose únicamente en los parámetros de sus creencias iniciales que determinan su capacidad operativa.

El agente robot\_light representa al robot de menor capacidad y mayor agilidad del sistema. Su base de creencias inicial se compone de:

- robot\_type(light): identificador del tipo de robot.
- max\_weight(10): capacidad máxima de carga en kilogramos.
- max\_size(1,1): dimensiones máximas del contenedor que puede manipular (1×1 celdas

lógicas).

- `state(idle)`: estado inicial de disponibilidad.
- `carrying(none)`: indica que no transporta ningún contenedor al inicio.

El ciclo de trabajo se dispara al recibir la creencia `task(CId, ShelfId)` procedente del *scheduler*. Si el robot está en estado `idle`, ejecuta secuencialmente `move_to_container` → `pickup` → `move_to_shelf` → `drop_at`. Al finalizar notifica al *scheduler* y al *supervisor* del resultado mediante `.send`. Ante cualquier error del entorno, planes de fallo específicos por tipo limpian el estado del robot y notifican el incidente.

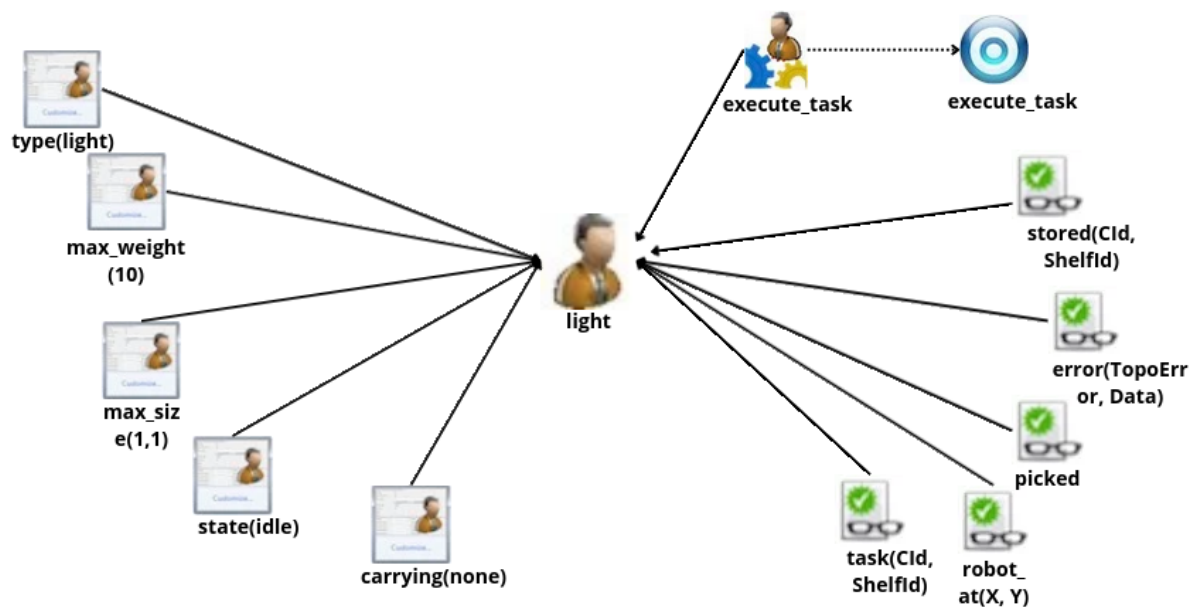


Fig. 4.6. Diagrama de agente: robot light.

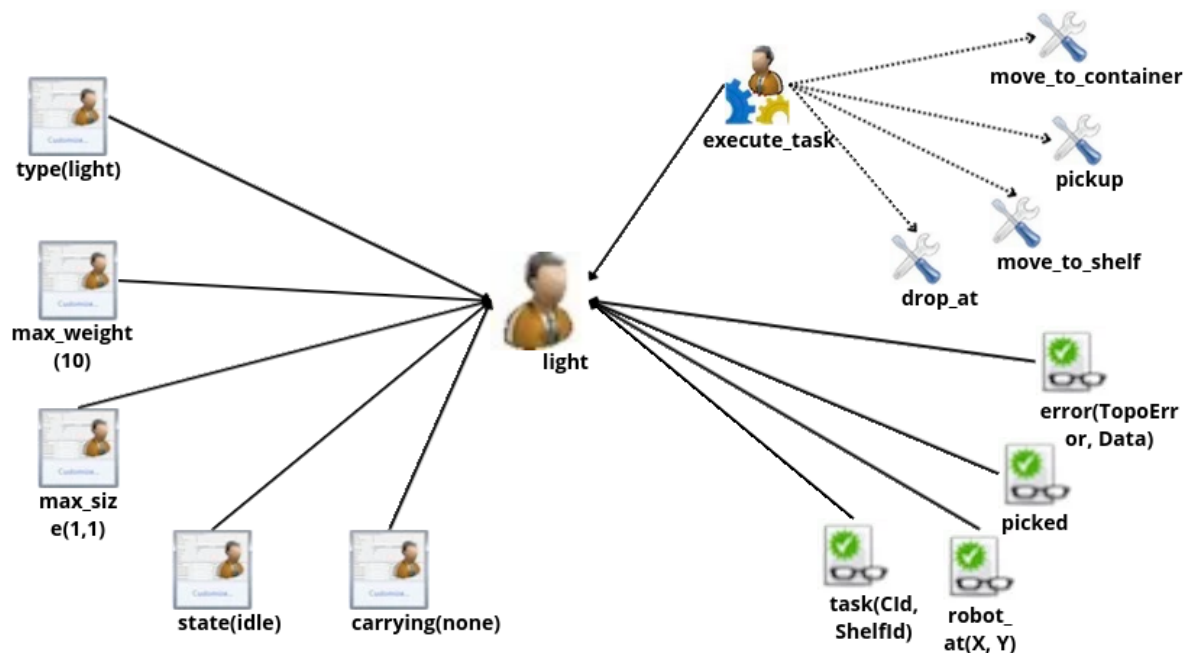


Fig. 4.7. Diagrama de tareas: robot light.

El agente `robot_medium` posee una base de creencias análoga a la del robot ligero, pero con parámetros que reflejan una mayor capacidad operativa:

- `robot_type(medium)`: identificador del tipo de robot.
- `max_weight(30)`: capacidad máxima de carga en kilogramos.
- `max_size(1,2)`: dimensiones máximas del contenedor que puede manipular (1×2 celdas lógicas).
- `state(idle)`: estado inicial de disponibilidad.
- `carrying(none)`: indica que no transporta ningún contenedor al inicio.

Su arquitectura de comportamiento es idéntica a la del robot ligero. Sus dimensiones de 1×2 celdas lo hacen más propenso a bloqueos en pasillos estrechos, por lo que los planes de error de navegación cobran especial relevancia en su operativa.

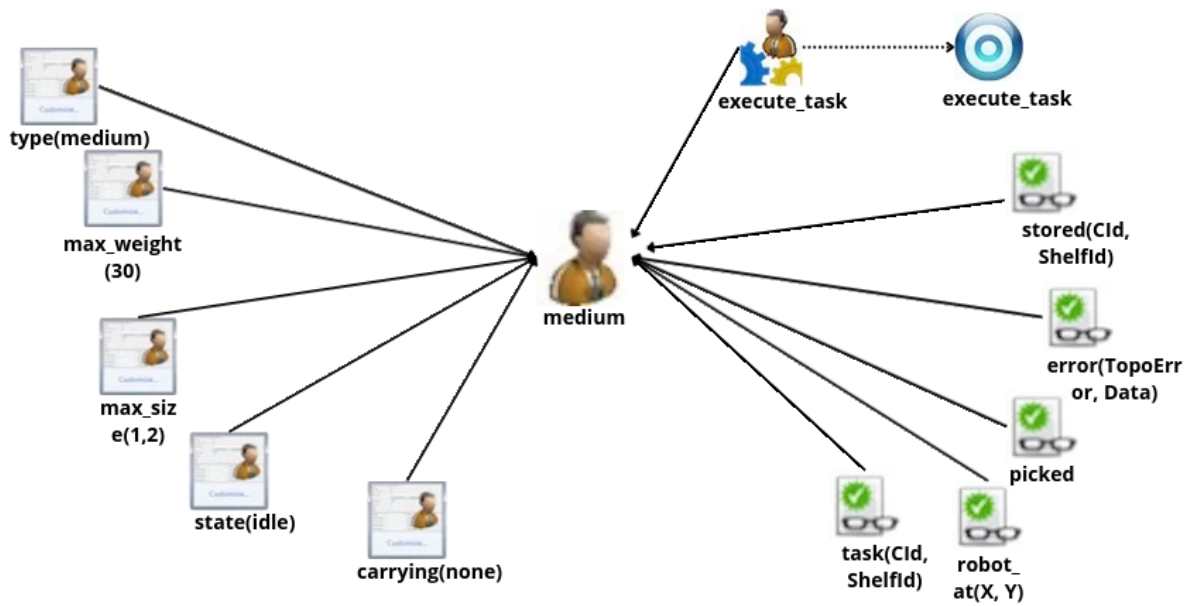


Fig. 4.8. Diagrama de agente: robot medium.

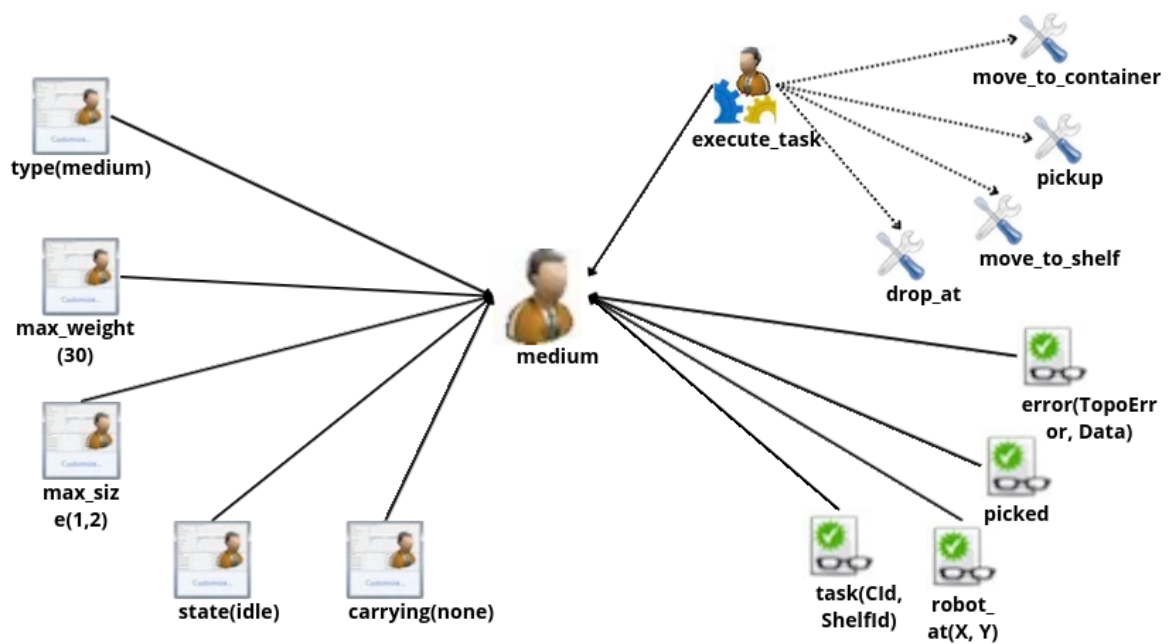


Fig. 4.9. Diagrama de tareas: robot medium.

El agente `robot_heavy` actúa como recurso crítico del sistema, capaz de manipular los contenedores de mayor peso y volumen. Sus creencias iniciales son:

- `robot_type(heavy)`: identificador del tipo de robot.
- `max_weight(100)`: capacidad máxima de carga en kilogramos.
- `max_size(2,3)`: dimensiones máximas del contenedor que puede manipular (2×3 celdas lógicas).
- `state(idle)`: estado inicial de disponibilidad.
- `carrying(none)`: indica que no transporta ningún contenedor al inicio.

Su ocupación de 6 celdas lógicas lo convierte en el agente más susceptible de causar bloqueos de ruta. Los planes de recuperación ante fallo incluyen una pausa de seguridad de 1,5 s para evitar que su volumen congele al resto de robots en los pasillos.

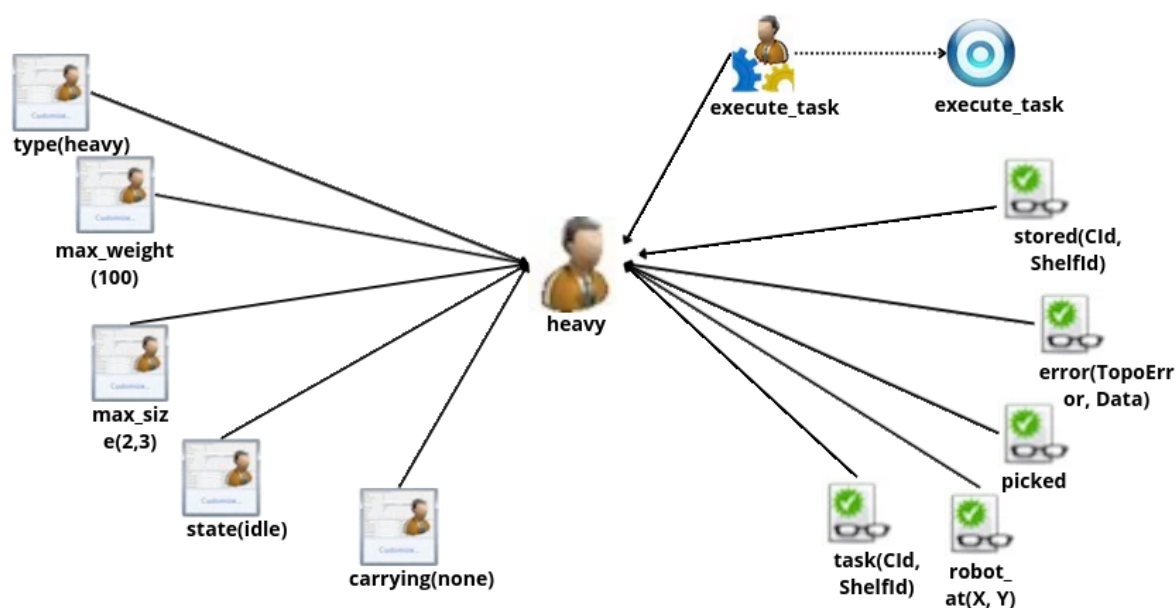


Fig. 4.10. Diagrama de agente: robot heavy.

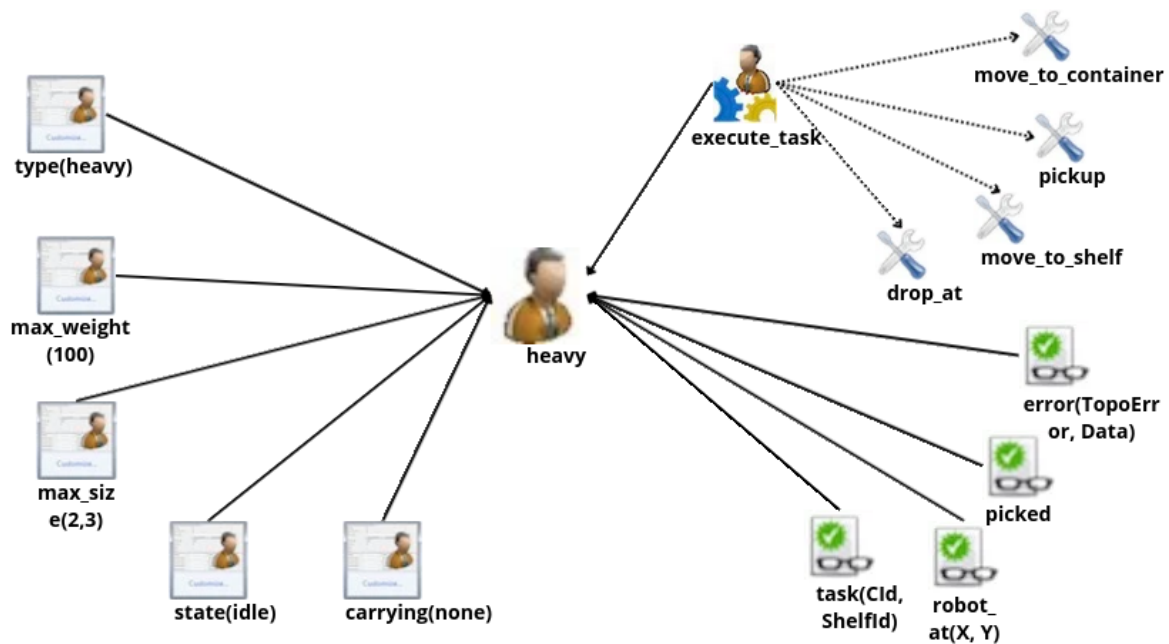


Fig. 4.11. Diagrama de tareas: robot heavy.

#### 4.6.2. Agente scheduler

El agente *scheduler* constituye el cerebro decisional del sistema. A diferencia de los robots, cuyas creencias reflejan su estado operacional local, las creencias del planificador almacenan el estado semántico global de todo el almacén.

Su base de creencias estática contiene las reglas de negocio de capacidad:

- `robot_capacity(...)`: tablas de capacidad máxima de carga y tamaño por tipo de robot.
- `robot_available(...)`: estado de disponibilidad de cada robot.

Sus creencias dinámicas incluyen contadores de operación global:

- `total_containers_received(0), total_task_assigned(0), pending_containers(0)`: métricas acumuladas para supervisión.
- `assigned(Robot, CId, ShelfId)`: creencia generada dinámicamente que permite al planificador saber en todo momento qué contenedor está siendo manipulado por qué robot, garantizando trazabilidad exacta.

El planificador reacciona principalmente a dos eventos: la aparición de nuevos contenedores

(+new\_container(CId)), que dispara el objetivo !assign\_shelf(CId), y la recepción de notificaciones de los robots (task\_failed, container\_stored), que actualizan la trazabilidad y relanza el ciclo de vida del contenedor si es necesario.

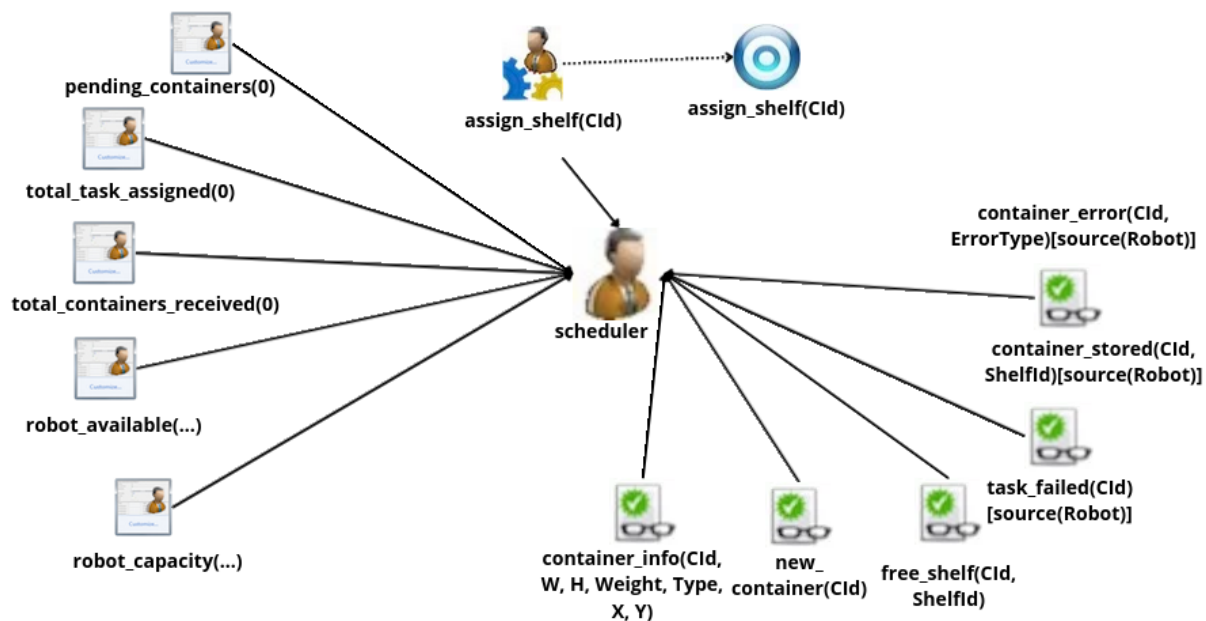


Fig. 4.12. Diagrama de agente: scheduler.

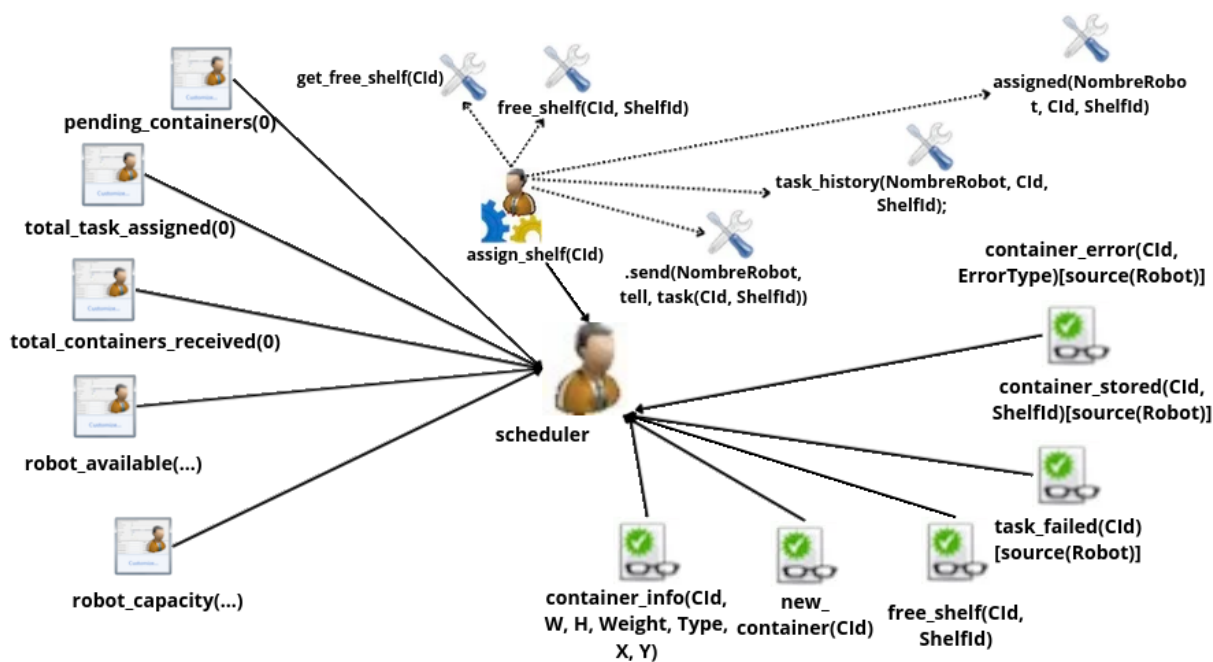


Fig. 4.13. Diagrama de tareas: scheduler.

### 4.6.3. Agente supervisor

El agente *supervisor* es una entidad puramente observadora del sistema. No impone objetivos ni emite órdenes a los demás agentes; su función es registrar métricas y reportarlas periódicamente.

Sus creencias iniciales están diseñadas para acumular contadores a lo largo de la ejecución:

- `total_received(0), total_stored(0), total_errors(0)`: contadores globales de contenedores recibidos, almacenados y errores producidos.
- `errors_by_type(...)`: clasificación de errores por tipo para análisis posterior.
- `robot_status(...)`: estado operacional de cada robot en tiempo real.
- `report_interval(30000)`: intervalo en milisegundos para la impresión periódica del estado global del SMA por consola.

El comportamiento del supervisor es puramente reactivo. Escucha eventos emitidos por los robots (`container_stored`, `container_error`, `robot_state_change`) y percepciones directas del entorno (`new_container`). Paralelamente, el objetivo iterativo `!stats_loop` lanza `!print_stats` cada 30 segundos, que recaba todos los datos usando `.count` y `.findall` para imprimirlos por consola de forma estructurada.

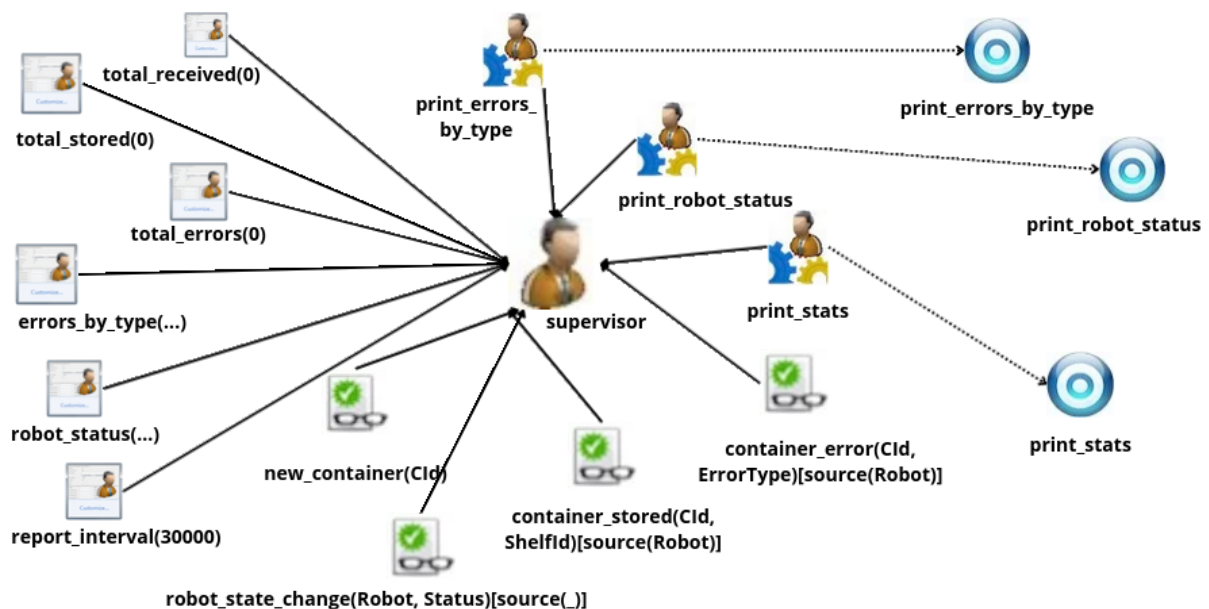


Fig. 4.14. Diagrama de agente: supervisor.



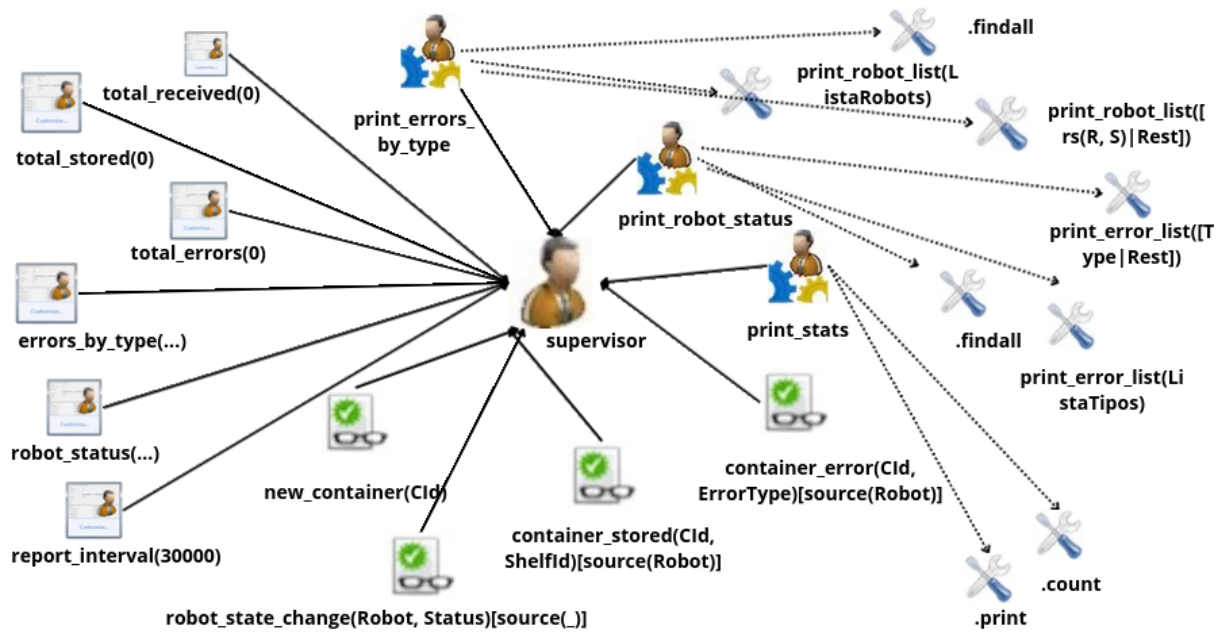


Fig. 4.15. Diagrama de tareas: supervisor.

## 5. Arquitectura: Patrón MVC (Modelo-Vista-Controlador)

El entorno del sistema *Warehouse* ha sido implementado siguiendo el patrón de diseño Modelo-Vista-Controlador (MVC), que separa la representación de la información, la lógica de negocio y la gestión de eventos en tres capas independientes. Esta separación facilita el mantenimiento del código y permite evolucionar cada capa sin afectar a las demás.

### 5.1. El Modelo: datos y estado del almacén

El modelo está constituido por las clases de datos Java ubicadas en el paquete `src/env/warehouse/`, que almacenan el estado del almacén y encapsulan sus reglas físicas. Los archivos principales son `Container.java`, `Robot.java`, `Shelf.java` y `CellType.java`.

Cada clase tiene una responsabilidad bien delimitada: `Container.java` almacena el identificador, las dimensiones, el peso, el tipo y la posición de cada unidad de carga; `Robot.java` mantiene la posición actual, el estado de ocupación y la referencia al contenedor transportado; `Shelf.java` gestiona la lógica de capacidad de almacenamiento mediante el método `canStore()`, que determina si una estantería puede aceptar un nuevo contenedor atendiendo a su volumen ocupado y peso acumulado; y `CellType.java` define la naturaleza de cada celda de la rejilla mediante un enumerado con siete tipos: `EMPTY` (pasillo vacío), `ENTRANCE` (zona de entrada de contenedores), `CLASSIFICATION` (zona de clasificación), `STORAGE` (área de almacenamiento genérica), `SHELF` (estantería), `BLOCKED` (celda inaccesible) y `ROBOT` (posición ocupada por un robot). El estado del almacén completo se almacena en una rejilla bidimensional `CellType[20][15]` y en tres mapas indexados por identificador: `containers`, `robots` y `shelves`.

### 5.2. La Vista: interfaz gráfica

La vista es responsable exclusivamente de la representación visual del estado del almacén en tiempo real. El archivo clave es `WarehouseView.java`, que utiliza la librería Swing de Java para

renderizar la interfaz gráfica.

Su única responsabilidad es consultar el estado actual del Modelo y dibujarlo: celdas con sus tipos y colores, robots con su posición y estado de ocupación, contenedores con su identificador y color según categoría, y estanterías con su porcentaje de ocupación. La vista no toma ninguna decisión lógica ni modifica el estado del sistema; es un observador pasivo que se repinta cada vez que el Controlador lo notifica mediante una llamada a `repaint()`.

### 5.3. El Controlador: artefacto y agentes

En el patrón MVC clásico, el Controlador responde a eventos externos (normalmente de usuario) y actualiza el Modelo en consecuencia. En un SMA, este rol lo desempeñan los agentes BDI: en lugar de eventos de interfaz de usuario, reaccionan a percepciones del entorno, razonan sobre ellas y emiten acciones que modifican el estado del sistema. El controlador está por tanto dividido en dos niveles complementarios.

El controlador físico (`WarehouseArtifact.java`) actúa como puente entre los agentes y el Modelo, implementando la API de entorno nativa de Jason mediante la extensión de Jason: `jason.environment.Environment`. Recibe las acciones emitidas por los agentes —`move_to`, `pickup`, `store`, `get_free_shelf`, entre otras—, ejecuta la lógica física correspondiente (incluyendo el cálculo de rutas mediante búsqueda en anchura, BFS), modifica el Modelo y notifica a la Vista para que se repinte. Es también el responsable de generar los perceptos que los agentes recibirán en su siguiente ciclo de razonamiento.

Los controladores lógicos (archivos `.asl`) son los cinco agentes del sistema: `scheduler`, `supervisor`, `robot_light`, `robot_medium` y `robot_heavy`. Cada uno implementa su ciclo de razonamiento BDI de forma autónoma: percibe el estado del entorno a través de los perceptos que el artefacto le entrega, delibera sobre sus creencias e intenciones, y emite las acciones correspondientes. No existe un bucle de control centralizado externo; el comportamiento global emerge de la interacción entre agentes.

El flujo MVC completo, desde la aparición de un nuevo contenedor hasta su almacenamiento, se desarrolla como sigue:

- (1) El artefacto genera un nuevo contenedor y actualiza el Modelo con su posición y propiedades.
- (2) La Vista dibuja el contenedor en la rejilla.
- (3) El artefacto emite el percepto `new_container(CId)` a todos los agentes del sistema; es el

agente `scheduler` quien reacciona a él mediante su plan `+new_container(CId)`.

- (4) El `scheduler` solicita información del contenedor (`get_container_info`), lo clasifica, determina el robot apropiado y la estantería óptima, y envía la acción `task(CId, ShelfId)` al robot correspondiente.
- (5) El robot ejecuta su plan: navega hasta el contenedor (`move_to_container`), lo recoge (`pickup`), navega hasta la estantería (`move_to_shelf`) y lo deposita (`drop_at`). Cada una de estas acciones modifica el Modelo a través del artefacto.
- (6) La Vista se actualiza en cada paso, mostrando al robot desplazarse por la rejilla con el contenedor a cuestas hasta su destino final.

## 6. Implementación de la solución

El desarrollo de esta primera fase del proyecto se ha centrado en evolucionar una arquitectura base hacia un Sistema Multiagente completo, reactivo y robusto, capaz de gestionar la logística de un almacén automatizado. A continuación se detalla, por componente, qué existía en la versión base, qué problemas presentaba y qué modificaciones se introdujeron para resolverlos.

### 6.1. El Entorno Java

El entorno de simulación actúa como puente entre el plano físico del almacén y la percepción de los agentes. Se han introducido cambios significativos para optimizar la concurrencia y la detección de colisiones.

**Modelo de la cuadrícula y tipos de celda:** El almacén se representa como una matriz 2D donde cada celda tiene un tipo lógico que determina su función. Los tipos definidos en el enum `CellType` son: `EMPTY`, `ENTRANCE`, `CLASSIFICATION`, `STORAGE`, `SHELF`, `BLOCKED` y `ROBOT`. Esta tipificación permite que los algoritmos de navegación y asignación tomen decisiones espaciales sin acceder directamente al estado físico del mapa.

**Generación dinámica de contenedores:** La lógica de *spawn* se modificó para evitar cuellos de botella en la entrada. Los contenedores se generan aleatoriamente en celdas catalogadas como `ENTRANCE` que se encuentren libres en el momento de la generación, garantizando un flujo de entrada paralelo y evitando que todos los contenedores aparezcan en el mismo punto.

**Simplificación del *footprint* a  $1 \times 1$  en el grid:** Aunque los contenedores poseen dimensiones físicas reales ( $W, H$ ) utilizadas como *metadata* para la clasificación lógica, su representación espacial en la cuadrícula se ha simplificado a  $1 \times 1$ . Esto estandariza los algoritmos de movimiento y evita la complejidad de gestionar colisiones con áreas variables.

**Ciclo de vida del contenedor:** Se ha modelado un ciclo de vida estricto: *spawn* (aparición en `ENTRANCE`), *pickup* (recogida por un robot y bloqueo en memoria), transporte y *drop* (destrucción visual y persistencia en la estantería). La destrucción del contenedor del mapa se realiza correctamente en cada fase para evitar estados inconsistentes.

**Navegación BFS con reserva de destinos:** El algoritmo de búsqueda en anchura (BFS) calcula el camino libre más corto desde la posición del robot hasta el destino. Se añadió la reserva de destinos activos (`activeDestinations`) para evitar que dos robots intenten ocupar la misma celda simultáneamente. Se corrigió un *deadlock* donde los destinos quedaban permanentemente bloqueados: la clave del mapa de reservas pasó a ser la coordenada (X,Y) en lugar del nombre del agente, garantizando una liberación correcta en el bloque `finally` de `executeMoveTo`.

**Acciones `move_to_container` y `move_to_shelf`:** Se implementaron como acciones Java específicas con adyacencia dinámica. Calculan la casilla adyacente óptima al destino filtrando robots en tránsito y contenedores adyacentes, de modo que el robot siempre se posiciona en la celda perimetral más próxima sin solaparse con otros agentes.

**Refactorización de `getAdyacentes` a `Container.java`:** La lógica para calcular las casillas adyacentes de un contenedor se movió de `WarehouseArtifact` a la entidad `Container.java`, respetando los principios de alta cohesión y responsabilidad única.

**Asignación de estanterías por niveles de peso (`findBestShelf`):** Se implementó una zonificación inteligente. El método busca estanterías en zonas preferentes según el peso del contenedor: ligeros en `shelf_1` a `shelf_4`, medianos en `shelf_5` a `shelf_7` y pesados en `shelf_8` y `shelf_9`. Si las zonas prioritarias están llenas, se ejecuta una búsqueda *fallback* por porcentaje de ocupación (`getOccupancyPercentage()`) para evitar inanición.

**Migración de API deprecada:** Para la generación dinámica de percepciones hacia Jason, se sustituyó el uso obsoleto de `Literal.parseLiteral` por la API moderna `ASSyntax.parseLiteral`.

## 6.2. El Agente Scheduler

El planificador (`scheduler.asl`) es el núcleo coordinador del sistema. Se ha refactorizado para abandonar heurísticas simples y adoptar un modelo de asignación multidimensional robusto.

**Detección de nuevos contenedores:** El agente reacciona de forma asíncrona a la percepción `new_container(CId)`, emitida globalmente por el entorno al aparecer un contenedor. Este evento dispara el objetivo intermedio `!process_new_container(CId)`, que a su vez ejecuta la acción `get_container_info(CId)` para obtener las propiedades físicas del contenedor: dimensiones, peso, tipo y posición. El uso de un *goal* intermedio, en lugar de procesar directamente en el *trigger* de creencia, es deliberado: permite que el plan de fallo `!process_new_container` capture situaciones en las que el contenedor ha sido aplastado entre su aparición y su procesamiento, evitando que Jason quede con una intención irrecuperable.

**Clasificación por peso, tamaño y tipo:** Se superó el modelo condicional básico. Ahora se evalúan tres dimensiones independientes para la asignación: peso (*Weight*), anchura (*W*) y altura (*H*), generando creencias de categoría separadas (*container\_weight\_category*, *container\_size\_category*, *container\_type*) que guían tanto la elección del robot como la de la estantería.

**Listas de contenedores por categoría:** La base de creencias mantiene listas dinámicas (*containers\_heavy*, *containers\_medium*, *containers\_light*) que permiten conocer en todo momento la carga de trabajo pendiente por segmento operativo.

**Asignación de estanterías con reintentos:** Se introdujo un mecanismo de resiliencia. Si `!assign_shelf(CId)` falla por falta de espacio, el planificador ejecuta `.wait(5000)` y consume un intento. Se estipuló un límite estricto de 3 intentos (*shelf\_retries*) antes de catalogar la operación como fallida y notificar al supervisor.

**Asignación de robots con `elif` y trazabilidad:** La estructura de decisión se simplificó eliminando `if` anidados en favor de cláusulas `elif`, mejorando la legibilidad. Una vez despachada la tarea, el scheduler genera la creencia `assigned(Robot, CId, ShelfId)` y la persiste en un histórico `task_history(Robot, CId, ShelfId)`, garantizando trazabilidad completa desde la recepción hasta el almacenamiento.

**Activación de contadores en tiempo real:** Se implementaron creencias actualizables (*total\_containers\_received*, *total\_tasks\_assigned*) que varían atómicamente con cada iteración, proporcionando métricas en tiempo real al supervisor.

### 6.3. Los Agentes Robot

Los agentes físicos (*robot\_light*, *robot\_medium*, *robot\_heavy*) han sido rediseñados para maximizar su autonomía reactiva frente a imprevistos en el plano físico.

**Capacidades diferenciadas:** Cada perfil de robot tiene parámetros específicos en sus creencias iniciales. El pesado soporta hasta 100 kg y áreas de  $2 \times 3$  celdas; el ligero (10 kg,  $1 \times 1$ ) es el más ágil; y el mediano (30 kg,  $1 \times 2$ ) actúa como equilibrio entre capacidad y movilidad.

**Paso al modelo puramente reactivo:** Se eliminó el antiguo patrón *pull* donde los robots hacían `request_task`. Ahora el sistema usa un patrón *push*: los robots aguardan en estado `idle` hasta recibir explícitamente el mensaje `tell: task(CId, ShelfId)` del scheduler, eliminando *polling* innecesario y condiciones de carrera en la solicitud de tareas.

**Fix del *tweaking* visual:** Se corrigió un error de solapamiento donde los robots se posicionaban encima del contenedor al recogerlo. La lógica de `move_to_shelf` y `move_to_container` fue alterada para que el *drop* y el *pickup* se realicen siempre desde una celda adyacente: el ligero aproxima a (SX+2, SY-1), el mediano a (SX+1, SY-1) y el pesado a (SX, SY-1).

**Ciclo de ejecución de tareas:** La macro-tarea se atomizó en un flujo transaccional secuencial: `move_to_container(CId) → pickup(CId) → move_to_shelf(ShelfId) → drop_at(ShelfId)`. Cada fase está separada con pausas de seguridad (`.wait`) para garantizar que el entorno Java procese cada acción antes de lanzar la siguiente.

**Cola de tareas explícita con `!check_queue`:** Se abandonó el chequeo basado en el *trigger* `+state(idle)`. Ahora, al finalizar cada tarea, el robot ejecuta `!check_queue`, que comprueba si hay tareas encoladas pendientes y las procesa ordenadamente, evitando pérdidas de asignaciones recibidas mientras el robot estaba ocupado.

**Planes de error específicos por tipo:** Para cumplir con los principios BDI, los planes de contingencia diferencian entre errores físicos de contenedor (`container_too_heavy`, `container_too_big`) y errores espaciales de navegación (`destination_conflict`, `route_blocked`, `path_blocked`, `too_far`, `illegal_move`, `robot_not_found`). Ante un fallo en `!execute_task`, el plan `!execute_task` limpia el estado del robot (`-+carrying(none)`, `release_task`) y notifica al *scheduler* con `task_failed(CId)` para reasignación.

**Notificación directa al supervisor:** Se eliminó el *relay* a través del *scheduler*. Ahora los robots notifican simultáneamente a ambos agentes: `container_stored(CId, ShelfId)` y `container_error(CId, ErrorType)` se envían directamente al *scheduler* para trazabilidad y al supervisor para auditoría, preservando el `[source(Robot)]` correcto en ambos casos.

**Notificación de cambios de estado y *fix* del *source*:** Los robots anuncian sus transiciones de estado mediante `robot_state_change(Robot, Status)`. Se detectó y corrigió un *bug* donde un commit intermedio redirigía estas notificaciones al *scheduler* como proxy, haciendo que el supervisor recibiera `[source(scheduler)]` en lugar de `[source(robot_x)]`. El plan del supervisor usaba el *source* para validar la identidad del emisor (`[source(Robot)]`), por lo que nunca disparaba. La corrección fue doble: los robots volvieron a notificar directamente al supervisor y el plan del supervisor pasó a usar `[source(_)]` para aceptar cualquier origen.



## 6.4. El Agente Supervisor

El agente supervisor funciona como torre de control y auditoría del sistema. Su implementación garantiza la recopilación de telemetría y el cálculo de estadísticas operacionales.

**Monitorización de eventos:** Escucha pasivamente los flujos procedentes de los robots (`container_stored`, `container_error`, `robot_state_change`, `robot_error`) y las percepciones directas del entorno (`new_container`), registrando cada evento como un hecho individual en su base de creencias.

**Refactorización de contadores:** Se descartó la aritmética manual con variables temporales (`-+total_received(N+1)`), propensa a condiciones de carrera si múltiples eventos llegaban seguidos. Ahora cada evento añade un hecho individual (`container_received(CId)`, `container_stored_fact(CId, ShelfId)`, `error_occurred(CId, ErrorType)`) y los totales se calculan en el momento del reporte con `.count`, sin estado intermedio que mantener.

**Fix de sintaxis Jason:** Se resolvieron *bugs* críticos de *parsing* que impedían cargar el supervisor. El operador de asignación aritmética `is` no es válido en Jason 3.3.0. Se substituyó por `=`, que es el operador correcto para unificación y evaluación aritmética en AgentSpeak.

**Estadísticas periódicas:** Se implementó un bucle `!stats_loop` que emite un reporte cada 30 segundos calculando: tasa de éxito (`almacenados/recibidos × 100`), tasa de error (`errores/recibidos × 100`) y contenedores pendientes en proceso.

**Desglose de errores por tipo:** El supervisor no solo cuenta los errores totales, sino que los clasifica y deduplica, diferenciando entre errores de contenedor (peso/tamaño) y errores de navegación (ruta bloqueada), emitiendo un desglose estructurado en cada reporte.

**Consulta de estado en tiempo real con `askOne`:** Se substituyó la caché asíncrona de estados de robots por la primitiva de comunicación síncrona `.askOne`, que fuerza a cada robot a responder con su estado actual exacto en el momento de generar el reporte, eliminando desfases entre el estado real y el registrado.

## 6.5. Robustez y Gestión de Fallos

En sistemas industriales simulados, la consistencia ante excepciones es tan importante como el *camino feliz*.

**Mecánica de aplastamiento:** Si un robot pisa un contenedor ligero o múltiples entidades ocupan una coordenada no permitida, el método `doMoveTo` del entorno detecta el conflicto, elimina el contenedor del mapa y lo declara destruido. El entorno emite la percepción `container_broken(CId)` de forma global para que todos los agentes puedan reaccionar.

**Patrón *goal* intermedio `!process_new_container`:** Se encapsuló toda la lógica de procesamiento dentro de un *goal* explícito. Cuando un contenedor es aplastado entre su aparición y su procesamiento por el *scheduler*, la acción `get_container_info` falla dentro del *goal*, que a su vez falla controladamente activando el plan `-!process_new_container`. Este plan imprime un aviso y descarta el contenedor sin dejar la intención principal del agente en estado irrecuperable — algo que no era posible con el *trigger* de creencia `+new_container` directamente.

**Guards `container_broken` en el *pipeline* del *scheduler*:** Se añadieron comprobaciones explícitas (`not container_broken(CId)`) como contexto previo en los planes `!assign_shelf`, `!assign_robot` y `+free_shelf`, evitando que el planificador intente asignar o transportar un contenedor que ya no existe en el mapa.

**Notificación por agotamiento de reintentos:** Agotados los 3 intentos de `!assign_shelf`, el planificador notifica explícitamente al supervisor un error de tipo `almacen_lleno`, registrando el evento para su contabilización estadística.

**Condiciones de carrera identificadas:** Se identificaron y mitigaron diversas condiciones de carrera, como el intento de recoger un contenedor en el mismo ciclo en que otra rutina lo declara destruido. La atomicidad intrínseca del ciclo de razonamiento de Jason y los bloques `finally` en Java garantizan que el sistema sea resiliente ante estas situaciones, incluyendo la saturación total de las estanterías y el *deadlock* por congestión de entrada, que se comporta como un estado estable esperado dada la capacidad finita del almacén.

## 7. Pruebas y Resultados

El sistema fue validado mediante ejecuciones controladas de la simulación, observando el comportamiento del SMA a través de la interfaz gráfica y los reportes por consola emitidos por el agente *supervisor*. Al tratarse de un entorno simulado determinista en su lógica pero no en su temporización, las pruebas se diseñaron para cubrir tanto el flujo nominal como los escenarios de fallo más relevantes.

### 7.1. Escenario base: operación nominal

En condiciones normales, los contenedores aparecen aleatoriamente en las celdas ENTRANCE y son detectados inmediatamente por el *scheduler* a través del percepto `new_container(CId)`. El planificador clasifica cada contenedor, selecciona el robot adecuado y emite la tarea correspondiente. El robot recibe la asignación, navega hasta el contenedor, lo recoge y lo deposita en la estantería asignada.

Se verificó que los tres robots operan en paralelo sin colisiones: el mecanismo `activeDestinations` impide que dos robots reserven simultáneamente la misma celda destino, y la adyacencia dinámica garantiza que cada robot se posicione en la celda perimetral más cercana sin solaparse. El *supervisor* confirma la operación mediante el reporte periódico, que muestra tasas de éxito del 100% en escenarios sin saturación.

### 7.2. Escenario de aplastamiento de contenedor

Cuando un robot pisa un contenedor ligero, el entorno detecta la colisión, elimina el contenedor del mapa y emite la percepción global `container_broken(CId)`. Se verificó que el *scheduler* descarta el procesamiento del contenedor destruido gracias al guard `not container_broken(CId)` presente en los planes `!assign_shelf` y `!assign_robot`. El plan de fallo `!process_new_container` imprime un aviso por consola y libera la intención sin dejar al agente en estado irrecuperable.

### 7.3. Escenario de saturación del almacén

Con el almacén en máxima ocupación, las llamadas a `!assign_shelf` fallan repetidamente. El planificador ejecuta hasta 3 reintentos con una espera de 5 s entre intentos (`shelf_retries`); agotados estos, notifica al *supervisor* un error de tipo `no_shelf_space` mediante `container_error(CId, no_shelf_space)` y descarta el contenedor. El entorno continúa generando nuevos contenedores en las celdas ENTRANCE; al acumularse sin ser recogidos, los contenedores entrantes son aplastados por los robots en tránsito o por contenedores solapados, produciéndose una cascada de eventos `container_broken`. El sistema alcanza un estado estable de saturación donde los robots permanecen en `idle` y toda la actividad queda registrada en los reportes del *supervisor*, comportamiento esperado dada la capacidad finita del almacén.

### 7.4. Validación del agente supervisor

Los reportes emitidos cada 30 segundos por el *supervisor* mostraron coherencia con la actividad observada en la interfaz gráfica: los contadores de contenedores recibidos, almacenados y errores coincidieron en todas las ejecuciones. La consulta síncrona mediante `.askOne` garantizó que el estado de los robots reflejado en el reporte corresponde al estado real en el instante de la impresión, sin desfases producidos por cachés desactualizadas.

## 8. Conclusiones y trabajo futuro

Trabajar con Jason y el modelo BDI ha resultado ser una forma razonablemente natural de organizar este tipo de sistemas: la separación entre agentes con roles distintos (*scheduler*, *robots*, *supervisor*) encaja bien con el paradigma BDI y facilita tanto el diseño como la depuración. Todos los objetivos planteados para esta fase están implementados y funcionan en ejecuciones reales.

Las líneas de extensión más relevantes para fases posteriores son: la priorización dinámica de contenedores urgentes o frágiles en la cola del *scheduler*, la organización global de la carga, la introducción de un límite de reintentos en el robot ante situaciones de *shelf\_full* prolongado que lleve a un descarte de los contenedores que se sueltan en el pasillo, y la optimización de la asignación de tareas teniendo en cuenta la posición actual de cada robot para minimizar la distancia total recorrida. A nivel de infraestructura, la incorporación de persistencia de estado permitiría reanudar la operación del almacén tras un reinicio sin perder la trazabilidad de los contenedores en tránsito.

# Bibliografía

- [1] Rao, A. S., & Georgeff, M. P. (1995). BDI agents: From theory to practice. *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*.
- [2] Rao, A. S. (1996). AgentSpeak(L): BDI agents speak out in a logical computable language. *Proceedings of MAAMAW'96* (LNAI 1038). Springer.
- [3] Bordini, R. H., Hübner, J. F., & Wooldridge, M. (2007). *Programming multi-agent systems in AgentSpeak using Jason*. John Wiley & Sons.
- [4] Bordini, R. H., & Hübner, J. F. *Jason: A Java-based interpreter for an extended version of AgentSpeak*. Disponible en: <https://jasonlang.net>
- [5] Padgham, L., & Winikoff, M. (2002). Prometheus: A pragmatic methodology for engineering intelligent agents. *Workshop on Agent-Oriented Methodologies, OOPSLA 2002*.
- [6] Padgham, L., & Winikoff, M. (2004). *Developing intelligent agent systems: A practical guide*. John Wiley & Sons.